

Exercise – Understanding indirect inference

1. Aim of the exercise

The aim of this exercise is to apply indirect inference as a simulation-based estimation technique for recovering parameters in models with analytically intractable likelihood functions. Specifically, we focus on estimating the parameter of a moving average process using an auto regressive auxiliary model. Through repeated simulations, auxiliary model fitting, and optimization, we demonstrate how indirect inference can effectively approximate true model parameters and assess their statistical significance. Before proceeding with this application, it is recommended to first review the exercise on the theory of indirect inference.

2. True and auxiliary model setup for indirect inference

In this exercise, we assume that the true data-generating process is an MA(1) model, given by the form $y_t = \epsilon_t + \theta\epsilon_{t-1}$, where ϵ_t is white noise. Estimating the MA(1) parameter is often challenging due to the complexity of the likelihood function and the risk of non-invertibility (Osborn, 1976). To address these difficulties, we adopt an indirect inference approach, using an AR(1) model as an auxiliary representation. The AR(1) process is specified as $y_t = \beta y_{t-1} + \epsilon_t$, and the estimated coefficient β serves as a summary statistic to match simulated and observed data.

3. Set simulation parameters

Clear the workspace. Set the number of simulated datasets to 100. This means the model will generate 100 different datasets by repeatedly simulating data with different random errors. Averaging across these simulations reduces the impact of random variation in any single dataset, leading to more stable and reliable estimates. Specify the true value of the moving average parameter as 0.5. This is the target value we aim to recover through indirect inference. Specify the standard deviation of the error term as 1. These values represent the underlying structure of the model that generates the data. Set the length of each time series to 1000 observations, meaning each simulated dataset will contain 1000 data points. Finally, define a weight matrix, which in this case is simply the number 1. This matrix will later be used to calculate how far the simulated results deviate from the observed data when estimating the model parameters using indirect inference.

```
13 %% 3. Set simulation parameters
14
15 % 3.1. Clear workspace
16 clear;
17
18 % 3.2. Number of simulated datasets
19 N_sim = 100;
20
21 % 3.3. True MA(1) parameter (structural)
22 theta_true = 0.5;
23
24 % 3.4. Standard deviation of error term
25 sigma_true = 1;
```

```

26
27 % 3.5. Length of time series
28 T = 1000;
29
30 % 3.6. Weight matrix for objective function (scalar)
31 W = 1;

```

4. Simulate MA(1) process

This section simulates the MA(1) process. Begin by generating a sequence of random error terms from a normal distribution with a specified mean and standard deviation. These error terms form the basis of the time series. Next, initialize an empty vector to store the simulated values and fill it using a loop that applies the MA(1) formula: each value in the time series is calculated as the sum of the current error term and a scaled version of the previous error term, where the scaling factor is the true ma parameter. The result is a synthetic time series that reflects short-term dependencies, which will later be used to estimate parameters through indirect inference.

```

33 %% 4. Simulate MA(1) process
34
35 % 4.1. Generate error terms
36 epsilon = random("Normal",0,sigma_true,[T+1,1]);
37
38 % 4.2. Initialize time series
39 y = NaN(T,1);
40
41 % 4.3. Generate MA(1) data using theta_true
42 for t = 2:T+1
43     y(t-1) = epsilon(t)+theta_true*epsilon(t-1);
44 end

```

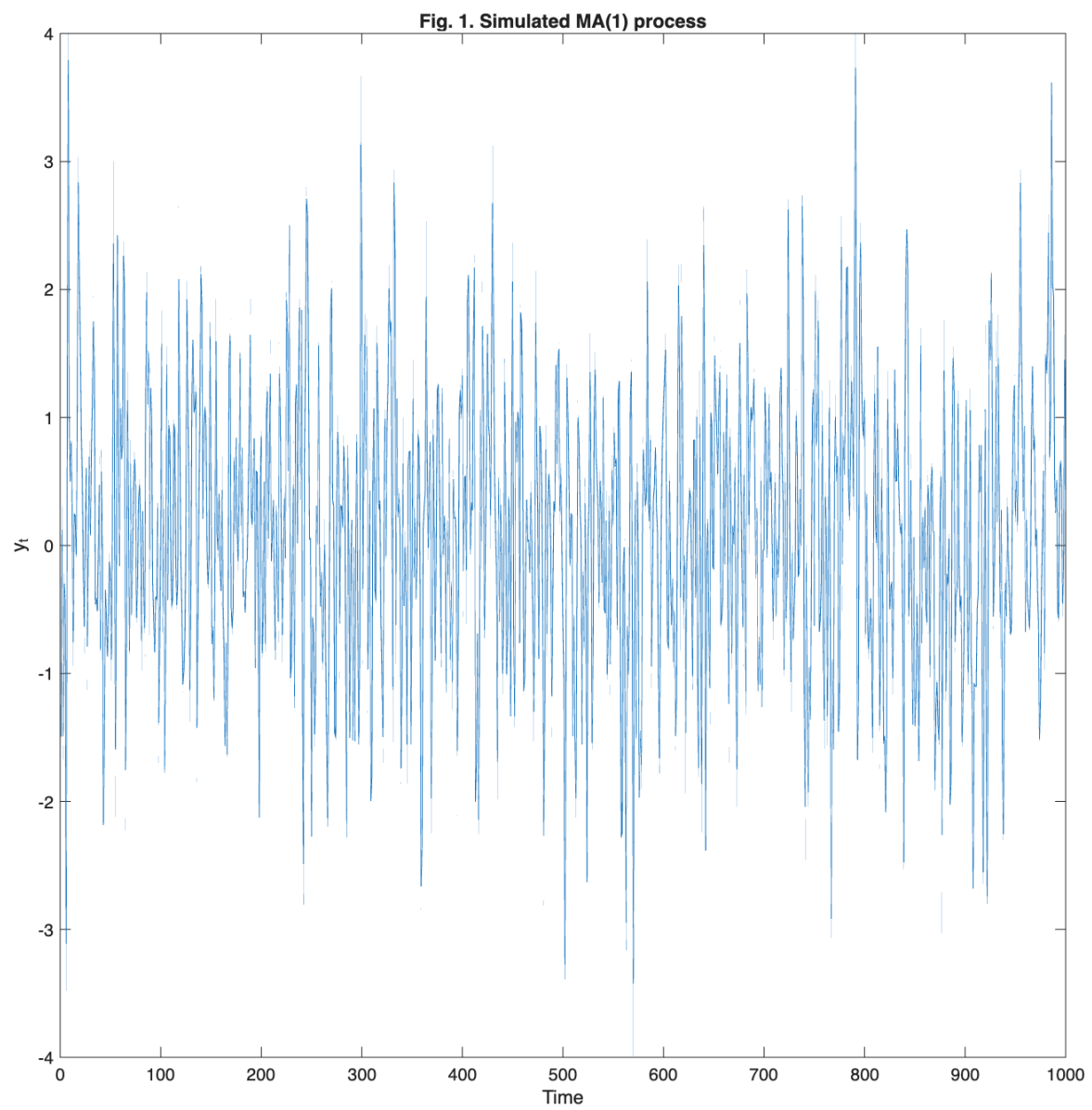
5. Plot simulated data

Create a visual representation of the simulated MA(1) time series.

```

46 %% 5. Plot simulated data
47
48 % Plot
49 figure('Position',[100 100 1000 1000]);
50 plot(y)
51 title('Fig. 1. Simulated MA(1) process');
52 xlabel('Time');
53 ylabel('y_t');

```



6. Define auxiliary model estimation function

Refer to the PDF file on the auxiliary model.

```
55 %% 6. Define auxiliary model estimation function
56 % See the function file auxiliary.m.
```

7. Estimate auxiliary model on simulated data

Estimate the parameter of the auxiliary model using observed data.

```
58 %% 7. Estimate auxiliary model on observed data
59
60 % AR(1) estimate from MA(1) data
61 beta_hat = auxiliary(y);
```

8. Define indirect inference function

We define an indirect inference function to minimize a distance measure between auxiliary estimates. The function has five input arguments: a candidate value for the structural parameter θ used to simulate data from the MA(1) model, the number of simulations to perform, the length of the time series, the diagonal weight matrix W used to compute the distance measure Q , and the auxiliary parameter $\hat{\beta}$ estimated from the observed data. For each simulation iteration, we generate data from the true data-generating process, in this case, the MA(1) model, using the candidate value of θ . We then estimate the auxiliary parameter β from each simulated dataset using the AR(1) model. The function computes the average of these simulated estimates, denoted $\tilde{\beta}(\theta)$, and calculates the distance measure Q . Since we aim to find a value of θ that makes the auxiliary estimates from simulated data as close as possible to those from the observed data, we define the objective function as: $Q(\theta) = (\hat{\beta} - \tilde{\beta}(\theta))'W(\hat{\beta} - \tilde{\beta}(\theta))$.

```
63 %% 8. Define indirect inference objective function
64 % See the function file objective.m.
```

9. Estimate the parameter of the complex model using indirect inference

This section performs the core estimation step in the indirect inference procedure, targeting the MA(1) parameter. It begins by specifying an initial guess for the parameter value, which serves as the starting point for the optimization algorithm. The optimization settings are configured using `optimoptions`, selecting the Sequential Quadratic Programming method and setting tolerances and iteration limits to control the precision and efficiency of the search. The `fmincon` function is then used to minimize the objective function, which measures the distance between the AR(1) estimate derived from the observed data and the average AR(1) estimate obtained from simulated datasets generated using candidate values of the MA(1) parameter. The result is an estimated value of θ that best aligns the simulated model with the empirical data, providing an indirect estimate of the underlying MA(1) process.

```
66 %% 9. Estimate the parameter of the complex model using ind. inf.
67
68 % 9.1. Initial guess for optimal theta
```

```

69 theta_initial = 0.5;
70
71 % 9.2. Optimization output
72 options = optimoptions('fmincon', ...
73     'Algorithm','sqp', ...
74     'Diagnostics','off', ...
75     'Display','iter-detailed', ...
76     'MaxIterations',100, ...
77     'MaxFunctionEvaluations',100, ...
78     'OptimalityTolerance',1e-4, ...
79     'StepTolerance',1e-4, ...
80     'SpecifyObjectiveGradient',false);
81
82 % 9.3. Optimize to find theta that minimizes the objective function Q
83 theta_estimated = fmincon(@(theta) ...
84     objective(theta,N_sim,T,W,beta_hat),theta_initial, ...
85     [],[],[],[],[],[],[],options);

```

10. Simulate auxiliary estimates for Wald test preparation

This section performs a Monte Carlo simulation to generate repeated estimates of an auxiliary model parameter, likely an AR(1) coefficient, using data simulated from an MA(1) process. Each iteration creates a new sequence of random error terms, constructs a time series based on the estimated MA(1) parameter, and then re-estimates the auxiliary parameter from the simulated data. The resulting collection of estimates is stored for later analysis.

```

87 %% 10. Simulated estimation of AR(1) coefficients via MA(1) model
88
89 % 10.1. Preallocate vector to store AR(1) estimates from simulations
90 beta_hat_sim = NaN(N_sim,1);
91
92 % 10.2. Simulate data using estimated theta and re-estimate AR(1)
93 for i = 1:N_sim
94     epsilon_sim = random("Normal",0,1,[T+1,1]);
95     y_sim = zeros(T,1);
96     for t = 2:T+1
97         y_sim(t-1) = epsilon_sim(t)+theta_estimated*epsilon_sim(t-1);
98     end
99     beta_hat_sim(i) = auxiliary(y_sim);
100 end

```

11. Compute Wald statistic and critical Value

We calculate and evaluate the Wald statistic to test whether the estimated MA(1) parameter is statistically different from the true value used to generate the data. To do this, we first compute the variance of the auxiliary AR(1) estimates obtained from the simulated datasets. Although these estimates come from an auxiliary model, their variability reflects the uncertainty in the estimation of the MA(1) parameter. This variance is used as the denominator in the Wald formula. The Wald statistic is then calculated by taking the squared difference between

the estimated and true MA(1) values, divided by the variance of the simulated estimates. The result is displayed for reference. Next, we define the degrees of freedom for the test, which is set to one since we are testing a single parameter. We then calculate the critical value from the chi-squared distribution at the 95 percent confidence level. This value serves as the threshold for determining statistical significance.

To interpret the result, we compare the calculated Wald statistic to the critical value. If the Wald statistic exceeds the critical value, we conclude that the estimated parameter is significantly different from the true value, and the code displays a message indicating that the null hypothesis is rejected. If the Wald statistic is less than or equal to the critical value, the difference is not statistically significant, and the code displays a message stating that the null hypothesis cannot be rejected. This outcome helps assess whether the estimation method has successfully recovered the true parameter.

```

102 %% 11. Compute Wald statistic and critical value
103
104 % 11.1. Variance of simulated AR(1) estimates
105 var_beta_hat = var(beta_hat_sim);
106
107 % 11.2. Wald statistic
108 wald_statistic = (theta_estimated - theta_true)^2 / var_beta_hat;
109
110 % 11.3. Display Wald statistic
111 disp(['Wald statistic: ', num2str(wald_statistic)]);
112
113 % 11.4. Degrees of freedom
114 df = 1;
115
116 % 11.5. Critical value from chi-squared distribution
117 chisq_critical_value = chi2inv(0.95, df);
118
119 % 11.6. Hypothesis test
120 if wald_statistic > chisq_critical_value
121     disp(['Reject the null hypothesis: ' ...
122         'The parameters are significantly different.']);
123 else
124     disp(['Fail to reject the null hypothesis: ' ...
125         'The parameters are not significantly different.']);
126 end

```

12. Simulation stability and the shape of the objective function

Following the diagnostic plot code, we evaluate how the objective function behaves across a range of candidate values for the parameter of the complex model. For each value in the theta grid, the code simulates multiple datasets using that candidate parameter, estimates the auxiliary AR(1) coefficient from each simulation, averages those estimates, and then computes the distance between this average and the auxiliary estimate obtained from the original data. The resulting values form the curve shown in the plot.

The clarity and smoothness of this curve depend heavily on the number of simulations used. If the number is too low, say, ten, the average auxiliary estimate becomes noisy due to random

variation in the simulated datasets. This noise carries over into the objective function, making the curve jagged and potentially misleading. By increasing the number of simulations to 100, 500, or more, the averaging process becomes more stable, reducing the influence of randomness and producing a cleaner, more reliable curve. This smoother shape helps the optimizer identify the best-fitting parameter more accurately and ensures that the indirect inference procedure yields meaningful results.

```

128 %% 12. Simulation stability and the shape of the objective function
129
130 % 12.1. Range of candidate theta values
131 theta_grid = linspace(0,1,100);
132
133 % 12.2. Preallocate for objective values
134 Q_values = zeros(size(theta_grid));
135
136 % 12.3. Compute the AR(1) estimate from original MA(1) data
137 beta_hat = auxiliary(y);
138
139 % 12.4. Loop over theta candidates
140 for i = 1:length(theta_grid)
141     Q_values(i) = objective(theta_grid(i),N_sim,T,W,beta_hat);
142 end

```

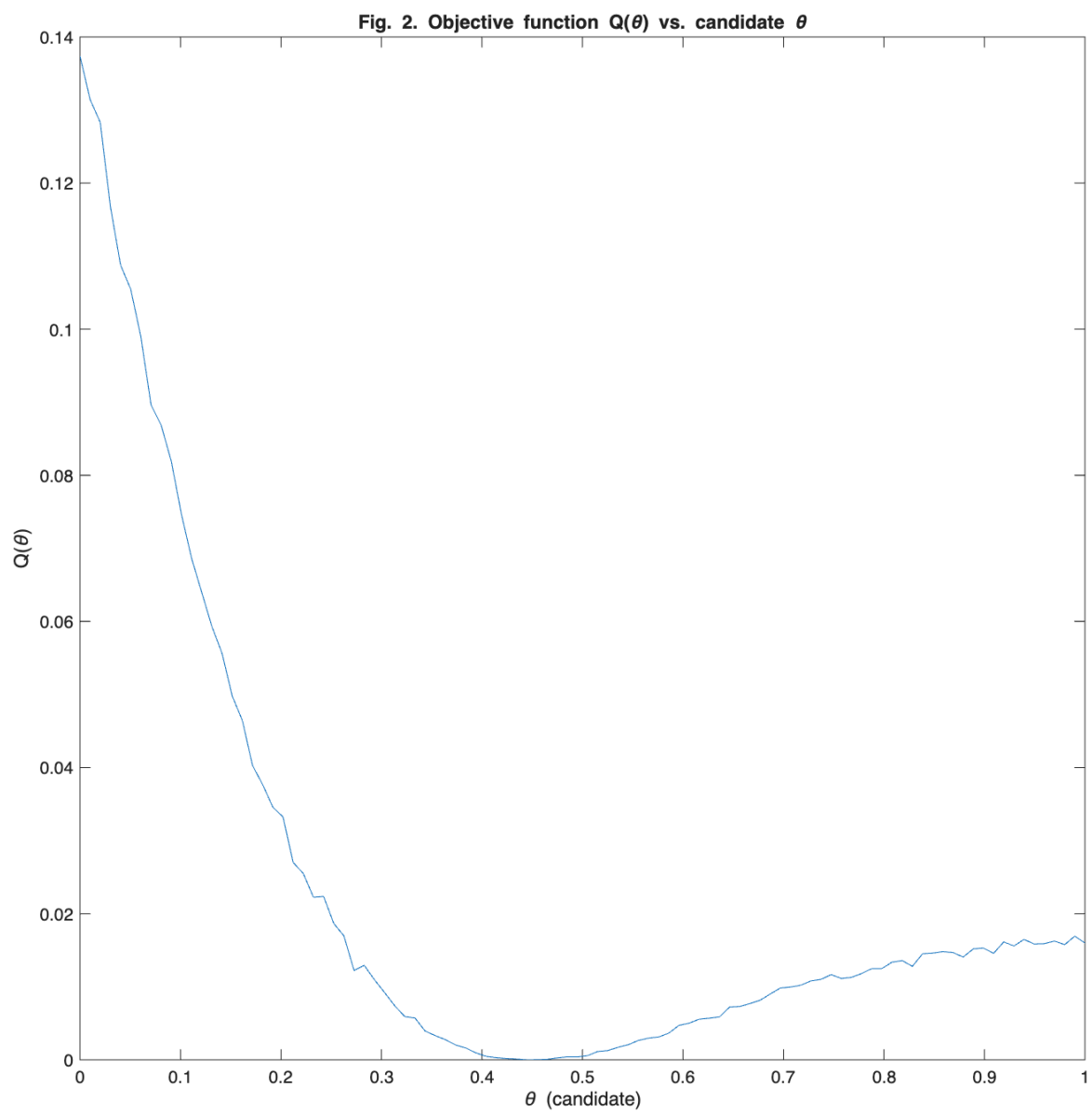
13. Plot

The plot illustrates the core mechanism of indirect inference. It shows how the objective function responds to different candidate values of the MA(1) parameter, with a clear minimum around 0.45. Although the true parameter used to generate the data is 0.5, the slight deviation in the estimated minimum may reflect simulation noise, sample variability, or the limitations of the auxiliary model. The smooth, convex shape of the curve suggests that a sufficiently large number of simulations was used, reducing randomness and enhancing the reliability of the estimation. Such diagnostic plots are essential in validating the stability and sensitivity of the indirect inference procedure, ensuring that the optimizer is guided by a well-behaved objective landscape.

```

144 %% 13. Plot
145
146 % Plot
147 figure('Position',[100 100 1000 1000]);
148 plot(theta_grid,Q_values);
149 title('Fig. 2. Objective function Q(\theta) vs. candidate \theta');
150 xlabel('\theta (candidate)');
151 ylabel('Q(\theta)');

```



14. Final notes

This file is prepared and copyrighted by Simonas Stravinskas and Tunga Kantarcı. This file and the accompanying MATLAB file are available on GitHub and can be accessed using this [link](#).